

Abstract

Agent-based models are a cornerstone in epidemiological research, enabling detailed simulations of disease spread by parameterizing aspects of the city, the infection process, and interventions. However, these simulations are computationally expensive, limiting the feasibility of counterfactual analyses across a large parameter space. In this project, we propose to run a large number of simulations upfront to generate a comprehensive dataset and then use machine learning models to learn a mapping from the parameter space to the SIR space. This map can later be used for rapid counterfactual analysis. We investigate several model architectures, including Feed Forward Networks, LSTMs, and XGBoost, and demonstrate that predicting the change (delta) in the SIR state yields superior performance compared to directly predicting the next state.

1 Introduction

Agent-based simulations are widely used in epidemiology to understand the spread of infections in cities. Traditional simulations involve parameterizing a city (e.g., distribution of location sizes, fraction of adults), the infection process (e.g., infectivity, recovery rate), and the interventions (e.g., lockdown start times, lockdown strength). Although these simulations offer rich insights, they are computationally intensive, making large-scale counterfactual analysis impractical.

To address this, we first generate a dataset by running a high number of simulations across a wide range of parameters. Our dataset spans 5776 unique configurations, achieved by varying six parameters:

- **Infectivity of the disease:** 2 values
- **Rate of recovery:** 2 values
- **Fraction of adults in the population:** 2 values
- **Distribution of location sizes:** 2 values
- **Lockdown start times:** 49 values
- **Lockdown strength:** 2 values

Since the simulation is Markovian, the next state is entirely determined by the current state and the interventions. Our objective is to learn a mapping from the current SIR state and interventions to the next SIR state. Alternatively, we consider predicting the change (delta) between consecutive states, which empirically leads to better performance.

2 Problem Formulation

Let $\mathbf{s}_t = (S_t^a, I_t^a, R_t^a, S_t^s, I_t^s, R_t^s)$ denote the state at time t , where S_t^i , I_t^i , and R_t^i represent the number of susceptible, infected, and recovered individuals, in class i , which represents adults (a) and students (s). The interventions (and other parameter inputs) at time t are denoted by $\mathbf{u}_t$. Our two proposed formulations are:

- (a) **Direct Prediction:** Learn a function

$$f : (\mathbf{s}_t, \mathbf{u}_t) \rightarrow \mathbf{s}_{t+1}.$$

- (b) **Delta Prediction:** Learn a function

$$f : (\mathbf{s}_t, \mathbf{u}_t) \rightarrow \Delta \mathbf{s}_t,$$

where $\Delta \mathbf{s}_t = \mathbf{s}_{t+1} - \mathbf{s}_t$.

In our experiments, the delta prediction approach yielded better results, as it focuses the learning task on the change in state rather than the absolute state values.

3 Data Generation and Preprocessing

The dataset was generated by running 10 agent-based simulation instances for each configuration and taking the mean trajectory as the ground truth.¹ Detailed configuration generation is handled by the `create_configs.py` script, while preprocessing is implemented in `data_loader.py` and `dataset.py`.² These scripts perform necessary tasks such as:

- Reading and parsing simulation outputs.
- Normalizing or scaling the data (we want S, I, R to be in the range [0, 1]).
- Splitting the dataset into training, validation, and test sets (75% train, 5% validation, 20% test).

4 Model Implementations

We explored three types of models for learning the simulation dynamics. The PyTorch and xgboost libraries were used for implementation. The models were trained using the Adam optimizer with a learning rate of 0.001 and a batch size of 64. The training process involved minimizing the Mean Squared Error (MSE) loss function:

$$\mathcal{L}(\mathbf{s}_{t+1}, \hat{\mathbf{s}}_{t+1}) = \frac{1}{N} \sum_{i=1}^N (\mathbf{s}_{t+1}^i - \hat{\mathbf{s}}_{t+1}^i)^2$$

where $\hat{\mathbf{s}}_{t+1}$ is the predicted state, and N is the number of samples in the batch. When we predict the delta, the loss function is modified to:

$$\mathcal{L}(\mathbf{s}_{t+1}, \mathbf{s}_t, \hat{\Delta \mathbf{s}}_t) = \frac{1}{N} \sum_{i=1}^N \left((\mathbf{s}_{t+1}^i - \mathbf{s}_t^i) - \hat{\Delta \mathbf{s}}_t^i \right)^2$$

4.1 Feed Forward Networks (FFN)

The FFN was designed to map the input $(\mathbf{s}_t, \mathbf{u}_t)$ directly to the next state $\mathbf{s}_{t+1}$ using the 2 methods described above. The architecture consisted of 4 hidden layers with ReLU activations, and a hidden size of 256. However, these networks tended to predict averaged out curves, failing to capture the dynamic behavior characteristic of SIR trajectories.



Figure 1: FFN model predictions for the SIR dynamics

¹Simulations were done using <https://github.com/RushilGupta4/AgentSim-julia>

²Refer to <https://github.com/DhrumanGupta/iml-midsem-project>

4.2 LSTMs

Given the sequential nature of the simulation data, LSTMs were employed to better capture temporal dependencies. Despite their capability to model sequences, similar to FFNs, LSTMs also resulted in smoothed outputs that did not resemble realistic SIR curves.

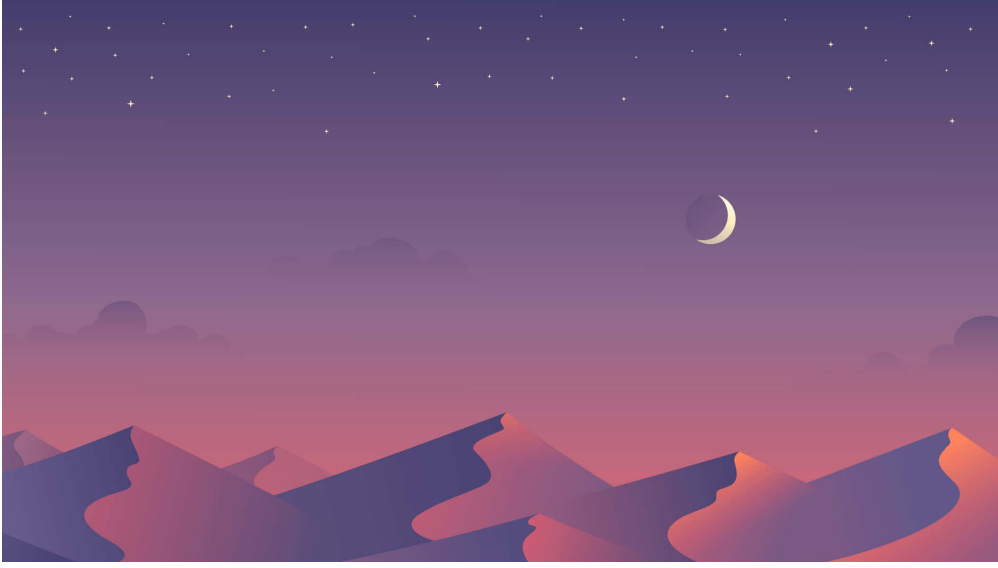


Figure 2: LSTM model predictions for the SIR dynamics

4.3 XGBoost

XGBoost, a gradient boosting framework, was applied in two different setups. Empirically, the delta prediction approach proved more effective. The model was able to give good predictions for the SIR trajectories for unseen input parameters.

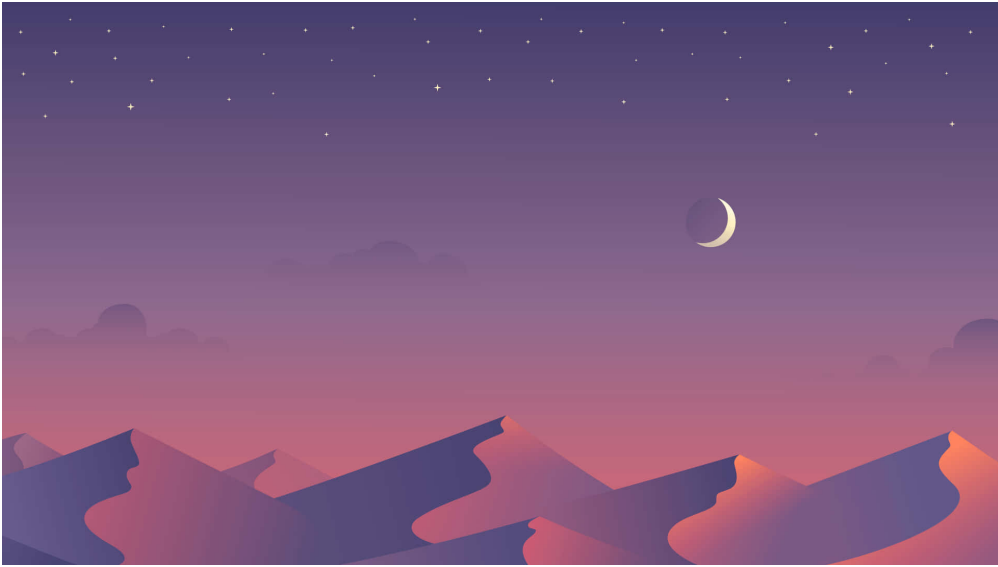


Figure 3: XGBoost model predictions for the SIR dynamics

5 Experimental Results

Our evaluation focused on comparing the predicted SIR trajectories to the ground truth obtained from the agent-based simulations. While both FFN and LSTM models resulted in predictions that were overly smoothed and deviated from expected SIR dynamics, the XGBoost model—particularly in the delta prediction setup—closely tracked the ground truth curves.

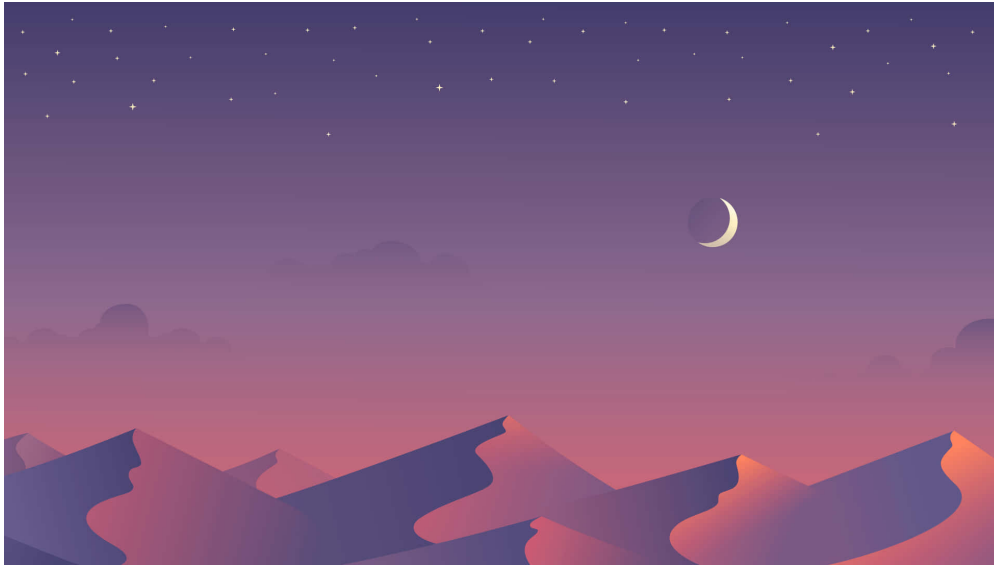


Figure 4: Comparison of model predictions with ground truth for the SIR dynamics for unseen input parameters

6 Conclusion and Future Work

In this project, we demonstrated that machine learning models can effectively learn the dynamics of expensive agent-based simulations in epidemiology. By predicting the state change rather than the absolute state, the XGBoost model achieved remarkable results in capturing the SIR dynamics. This surrogate modeling approach significantly reduces the computational cost of performing counterfactual analyses.

Here is what we plan on doing in the future:

- Refining the preprocessing pipeline to further improve model accuracy.
- Using larger datasets to enhance the model’s generalization capabilities.
- Use models that will give a distribution of s_{t+1} instead of a single value, allowing us to capture the distributions of the SIR dynamics.
- Extending the approach to more complex epidemiological models and additional intervention strategies.